



CSE488: Big Data Analytics [SUMMER 2026]

Term Project Report

Submitted by:

Student ID	Student Name	Contribution Percentage
Sheikh Sarafat Hossain	2022-3-60-109	30%
Md. Moon Rahman Nayem	2022-3-60-210	26%
Suraiya Akther Maisha	2022-3-60-071	24%
Rijia Parveen Raya	2022-3-60-192	20%

Submitted to:

Dr. Mohammad Rezwanul Huq

Associate Professor

Department of Computer Science and Engineering

East West University

Dataset Link: <https://github.com/sheikhsarafathossain/laptopchai.git>

1. Introduction

Selecting a suitable laptop is difficult because users need to consider several factors such as budget, processor, RAM, storage, GPU, display, operating system, weight, and intended workload. Traditional product search often depends on keywords and may fail to understand these requirements together or provide meaningful explanations. To address this problem, this project develops a RAG-based expert system for laptop recommendation that combines Big Data processing, semantic retrieval, approximate nearest-neighbor indexing, and LLM-based generation.

The system uses 1,875 laptop records, combining 1,303 public records and 572 scraped records. The data is cleaned, normalized, and deduplicated using Apache Spark and MinHash-LSH, reducing the dataset to 1,751 laptop records. Laptop specifications are converted into structured text, divided into 3,396 chunks, and transformed into 384-dimensional embeddings using all-MiniLM-L6-v2. The embeddings, text chunks, and associated metadata are stored in Parquet and subsequently ingested into ChromaDB, while multiple FAISS indexing methods are used for vector retrieval and comparison.

For a user's natural-language query, the system retrieves relevant laptops using semantic similarity and metadata constraints and provides a recommendation through an LLM based on the retrieved information. Finally, the pipeline is integrated into a Flask web application with product browsing, filtering, recommendation, and FAISS comparison features. The project demonstrates how Big Data technologies and RAG can be combined to build a practical laptop recommendation system.

2. Dataset and Data Collection

The project uses a combination of a publicly available Kaggle dataset and student-collected data scraped from multiple laptop-selling websites. The public dataset was obtained from the Laptop Price dataset on Kaggle, while the additional laptop records were collected through web scraping from different online sources.

Initially, the project contained 1,924 laptop records, including 1,303 records from the public dataset and 621 records collected through web scraping. After cleaning and validation of the scraped data, 572 valid scraped records were retained. These records were then merged with the 1,303 public records, resulting in a final dataset of 1,875 laptop records.

The dataset contains structured attributes such as Company, Product, TypeName, Inches, ScreenResolution, CPU, RAM, Memory, GPU, Operating System, Weight, and Price. These attributes were standardized during the data-cleaning process. The laptop specifications were also combined into a single textual description called `full_spec_text`. This text was later used for chunking, embedding generation, and semantic retrieval in the RAG pipeline.

3. Data Pre-processing

The Apache Spark processing pipeline was developed to clean, deduplicate, chunk, and generate embeddings from the collected laptop data. The merged dataset was processed using the Spark DataFrame API, where data types were standardized, unnecessary spaces were removed, and invalid records with null or non-positive prices and missing product information were filtered. A combined textual specification named `full_spec_text` was created from the laptop attributes, including company, product, category, display, processor, RAM, storage, graphics, operating system, weight, and price.

For near-duplicate detection, MinHash-LSH from Spark MLlib was applied to the specification text after tokenization, stopword removal, 3-gram generation, and CountVectorizer processing. Using a Jaccard distance threshold of 0.15, the system detected 159 near-duplicate pairs and removed 124 duplicate records, leaving 1,751 laptop records. The generated `full_spec_text` was then divided into overlapping chunks using a Spark UDF, with a chunk size of 40 words and an overlap of 10 words, producing 3,396 chunks.

Distributed embedding generation was performed using a Spark `pandas_udf` and the all-MiniLM-L6-v2 Sentence Transformer, producing 384-dimensional normalized embeddings in batches of 64. Finally, the generated embeddings, text chunks, and associated laptop metadata were stored in `laptops_embedded_chunks.parquet`, which was subsequently used for ingestion into ChromaDB.

4. Vector Database and ANN Retrieval

The generated laptop embeddings were used to build an efficient vector retrieval system. ChromaDB was used to store the embeddings, text chunks, and associated laptop metadata, while FAISS was used to implement and compare Approximate Nearest Neighbor (ANN) retrieval methods.

Five FAISS index types were considered in the retrieval pipeline: IndexFlatL2, IVF, Product Quantization (PQ), IVF+PQ, and HNSW. IndexFlatL2 provides an exact-search baseline, while IVF, PQ, IVF+PQ, and HNSW provide different approaches for improving retrieval efficiency and reducing search complexity.

The retrieval system uses the 384-dimensional embeddings generated by the all-MiniLM-L6-v2 Sentence Transformer. The application also supports metadata-based constraints such as price, brand, and laptop category alongside semantic similarity retrieval.

The implementation also provides a FAISS index comparison interface, allowing different retrieval approaches and their corresponding results to be examined.

5. RAG Chatbot Backend

A Retrieval-Augmented Generation (RAG) backend was developed to provide grounded laptop recommendations from natural-language user queries. The system converts the user query into an embedding and retrieves relevant laptop records using vector similarity. Metadata-based constraints can also be applied to refine the retrieved results according to requirements such as price, brand, category, RAM, CPU, and GPU.

The retrieved laptop information is used as context for the Large Language Model (LLM) to generate a recommendation and supporting explanation. This allows the recommendation to be based on the retrieved laptop specifications and helps keep the generated response relevant to the available dataset.

The system displays the generated AI recommendation together with the retrieved laptop results. Users can therefore inspect the recommended laptops along with their specifications, prices, and similarity scores.

6. Web Frontend

A web-based frontend was developed using Flask, HTML, and CSS to provide an interactive interface for the laptop recommendation system. The application contains three main views: Home, Products, and FAISS Comparison.

The Home page provides a natural-language search interface where users can describe their laptop requirements. The system displays an AI-generated recommendation along with the top retrieved laptops and their specifications, including CPU, RAM, storage, display, GPU, weight, and price.

The Products page provides access to the laptop catalog and supports filtering by brand and price range, as well as sorting by price. Each product is presented in a structured card containing its major specifications and price.

The FAISS Comparison page allows users to enter a search query and compare the retrieval results of different FAISS indexing methods. It displays the retrieved products along with the available comparison metrics, including Precision@5, Recall@5, and query latency.

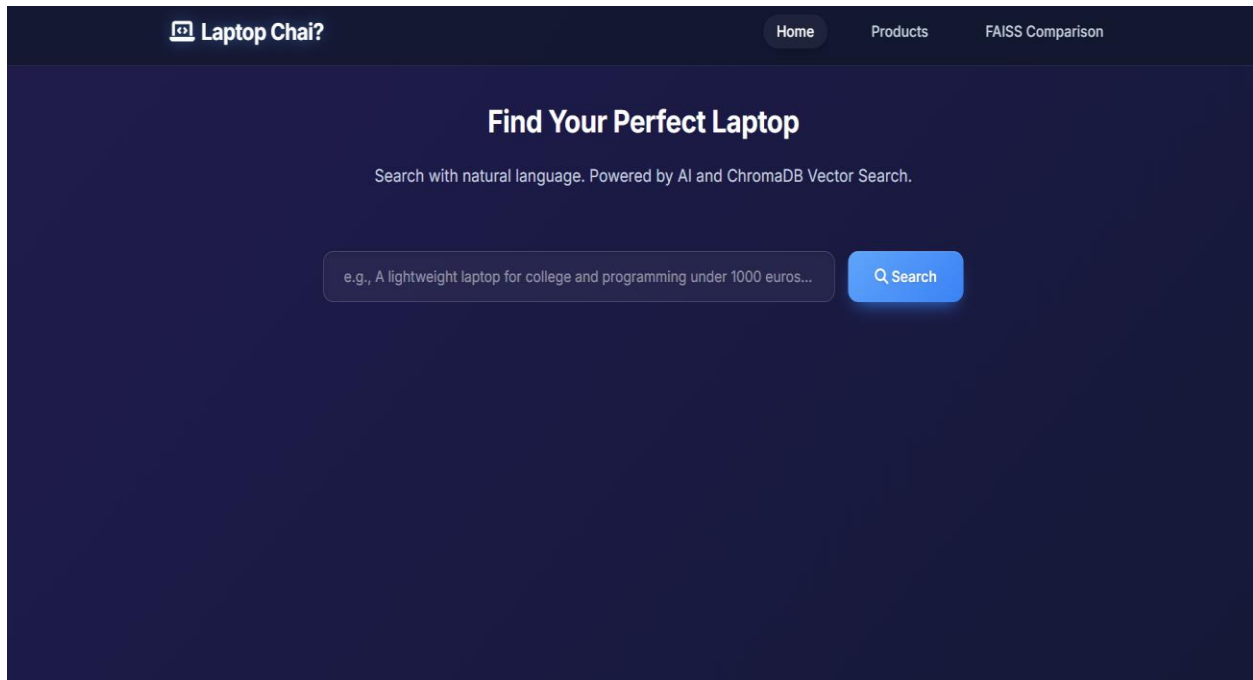


Figure 01: Home Page

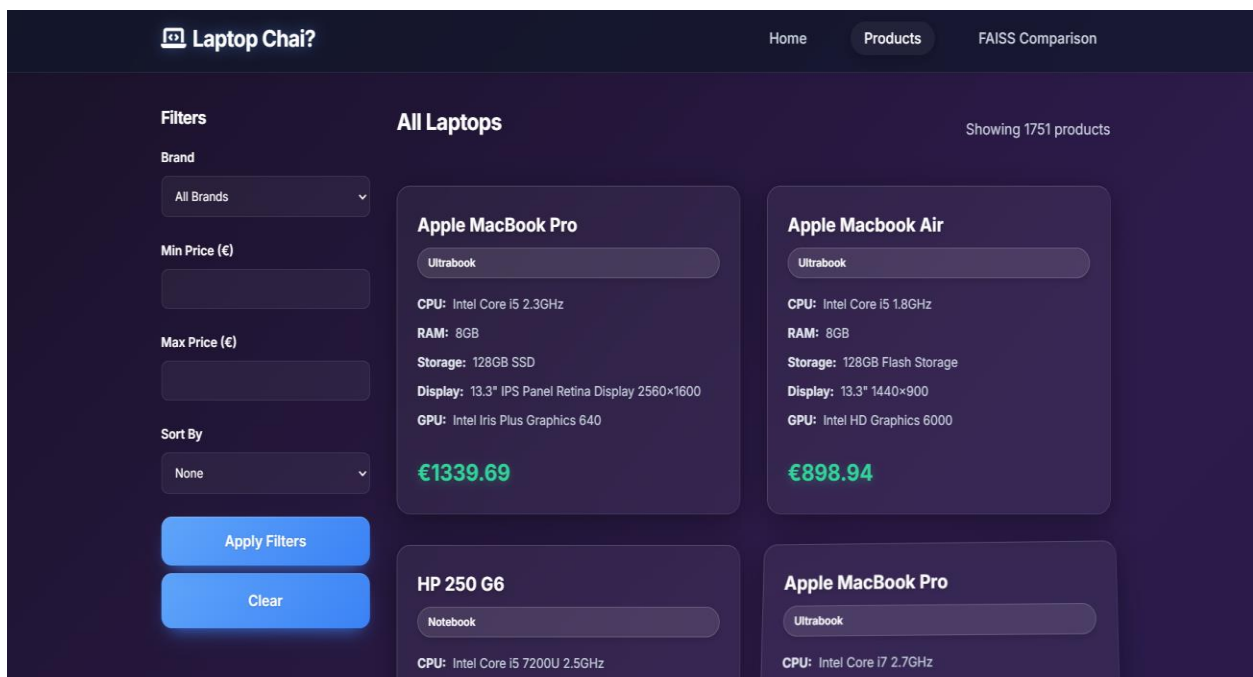


Figure 02: Products Page

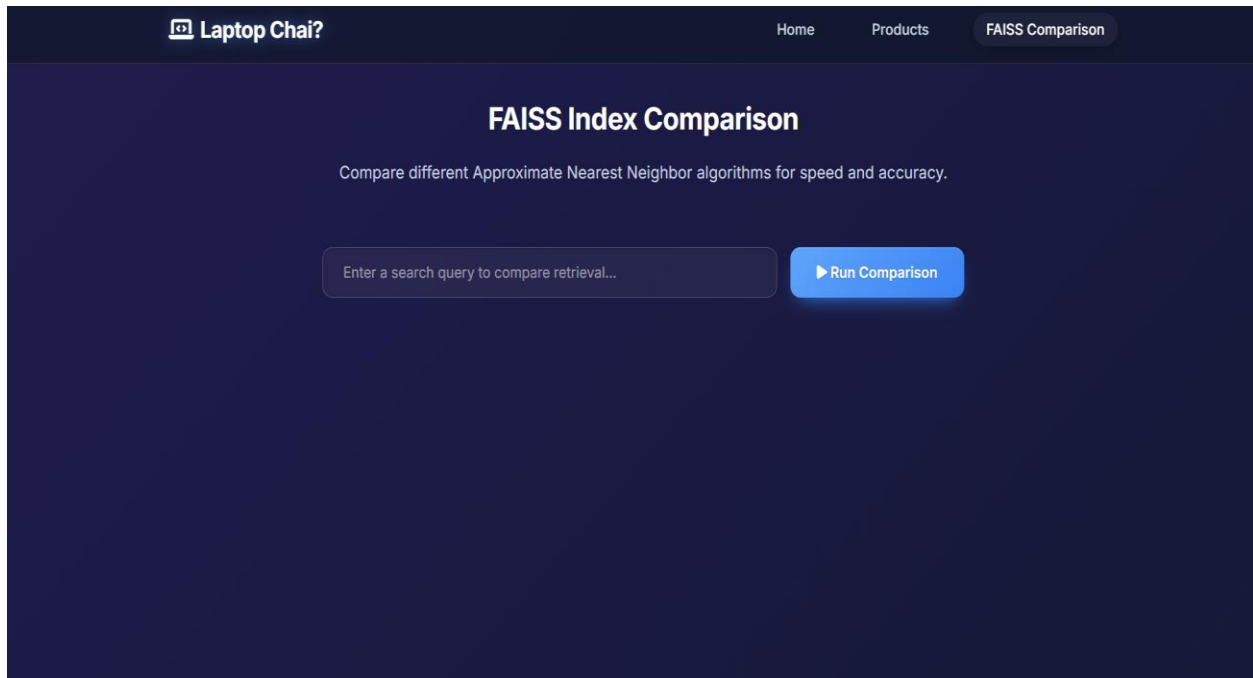


Figure 03: FAISS Comparison Page

7. Evaluation

The evaluation component of the project focuses on examining the retrieval performance and recommendation results of the developed system. The implemented FAISS comparison interface displays retrieval-related metrics such as Precision@5, Recall@5, and query latency for examining different indexing approaches.

The system also displays the retrieved laptop results along with their specifications and similarity scores, allowing the relevance of the retrieved products to be examined. In addition, the generated AI recommendations can be reviewed based on how well they correspond to the retrieved laptop information and the user's requirements.

This evaluation provides a basis for examining the effectiveness of the vector retrieval and recommendation components of the developed laptop recommendation system.

8. Discussion

The developed system demonstrates how Big Data processing, semantic retrieval, vector databases, ANN indexing, and RAG can be combined to build a practical laptop recommendation system. The dataset was collected from both a public Kaggle source and multiple scraped websites, providing a larger and more diverse set of laptop specifications. Apache Spark enabled systematic data cleaning, normalization, near-duplicate detection, text chunking, and distributed embedding generation.

The use of MinHash-LSH helped remove highly similar laptop records, while Sentence Transformer embeddings enabled semantic representation of laptop specifications. ChromaDB provided vector storage with associated metadata, and multiple FAISS indexing methods were incorporated to support efficient similarity retrieval. The hybrid retrieval approach further allows semantic similarity to work together with metadata constraints such as price, brand, and category. The Flask-based interface makes the system practical for users by providing natural-language search, product filtering, AI recommendations, and FAISS comparison. Overall, the combination of retrieval and LLM-based generation provides a more informative recommendation experience than simple keyword-based product search.

9. Conclusion

This project developed a RAG-based laptop recommendation system by integrating Big Data processing, semantic embeddings, vector databases, ANN retrieval, and LLM-based recommendation. A dataset of 1,875 laptop records was cleaned and deduplicated using Apache Spark and MinHash-LSH, resulting in 1,751 records. The laptop specifications were converted into textual descriptions, divided into 3,396 chunks, and transformed into 384-dimensional embeddings using the all-MiniLM-L6-v2 Sentence Transformer.

The generated embeddings and metadata were stored in Parquet and ingested into ChromaDB, while multiple FAISS indexing methods were incorporated for vector retrieval. The system also combines semantic retrieval with metadata-based constraints and provides recommendations through an LLM. Finally, the complete pipeline was integrated into a Flask web application with natural-language search, product browsing, filtering, AI recommendations, and FAISS comparison.

Overall, the project demonstrates the practical use of Big Data and RAG technologies to create a scalable and user-oriented laptop recommendation system.